Introduction to Robotics: ROS 2 Lab. 4

ROS 2 Gazebo and Rviz

Professor: Pietro Falco
Tutor: Alessandro Adami, email: adamialess@dei.unipd.it

Department of Information Engineering (DEI)
Università degli studi di Padova

April 1, 2026

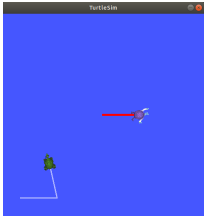
1. TurtleSim vs. Real Robots
2. ROS 2 on Real Hardware
3. Gazebo Simulation - Introduction
4. Gazebo: The Interface
5. Populating the World
6. Deep Dive: The 4 Key Robot Sensors
7. Lab Task: Gazebo Velocity Control
8. Inside the .world File
9. Beyond the Defaults: Custom Worlds
10. Launch Files
11. Advanced Spawning vs. Static Worlds
12. Hands-On Challenges

TurtleSim vs. Real Robots

Don't be fooled by the cute icon! **TurtleSim** is a 2D mathematical model of a real-world **TurtleBot**.

- **Kinematics**: Both use "Differential Drive" logic (two wheels moving independently).
- **Communication**: Both "listen" to the `geometry_msgs/Twist` message to move.
- **Feedback**: Both "publish" their **Pose** (position and heading).

The Simulation (TurtleSim)



Ideal, 2D, no physics.

The Reality (TurtleBot3)

Real-world physics, friction, and sensors.

Why start with the Turtle?

By mastering the Turtle, you are learning how to control a **Differential Drive** robot. The logic you used to draw a circle with the turtle is exactly the same logic used to navigate a robot on a laboratory floor.

Now that we understand the math behind the movement, we need to add Physics and 3D Space.

Feature Comparison

Feature	TurtleSim	Simulation (Gazebo)/Real World
Dimensions	2D (x, y, θ)	3D $(x, y, z$ + Roll, Pitch, Yaw)
Physics	None (Instantaneous)	Gravity, Friction, Torque
Sensors	Position only	Lidar, Cameras, Depth Sensors
Environment	Empty Blue Box	Realistic Worlds (Walls, Obstacles)

Complexity

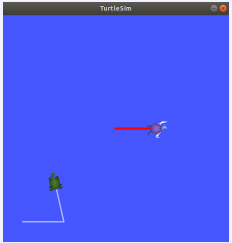
In Gazebo, if you drive your robot too fast into a wall, it will bounce or flip over. This is where we test **Perception** and **Obstacle Avoidance**.

Key Insight

The **Controller Node** you wrote in Lab 3 doesn't know the difference. It sends the same `linear.x` and `angular.z` to both!

Don't be fooled by the cute icon! **TurtleSim** is a 2D mathematical abstraction of a real-world **Differential Drive** robot, like the TurtleBot3.

TurtleSim (2D)



Abstract 2D environment

- Idealized movement.
- No gravity or friction.
- Perfect sensor data.

TurtleBot4 (3D / Real)

Physical robot in 3D space

- Real-world physics.
- Complex sensors (Lidar/IMU).
- Identical `Twist` interface.

ROS 2 on Real Hardware

The ultimate goal of ROS 2 is to run on physical hardware. Because of the Middleware architecture, the transition from Gazebo to a real robot is seamless.

- **Same Code, Different Driver**: Your `turtle_controller` node doesn't change. You simply replace the "Simulation Plugin" with a "Hardware Driver."
- **Hardware Abstraction**: The `/cmd_vel` topic is the universal language for moving robots, whether they are virtual or made of steel.

Deployment

On a real robot, the ROS 2 Graph usually spans multiple devices: a Workstation for visualization (RViz) and an On-board Computer (Raspberry Pi/Jetson) for real-time control.

Moving to real hardware introduces new challenges that were "perfect" in simulation.

- **Network Latency**: WiFi drops or delays can affect your control loop.
- **Sensor Noise**: Real Lidars and Encoders are not 100% accurate; they have "noise" and drift.
- **Battery & Power**: Real robots have limited energy and voltage drops under heavy load.

Handling the Gap

Problem	Simulation	Real World Solution
Slippage	Ideal Friction	Odometry Correction (EKF/SLAM)
Safety	Simple Collision	Watchdog Timers & Emergency Stops
Connectivity	Localhost	ROS_DOMAIN_ID & DDS Tuning

Gazebo Simulation - Introduction

While **Turtlesim** is great for learning ROS 2 "plumbing," **Gazebo** is a professional-grade 3D simulator used to test robots in realistic environments.

- **Physics Engine**: Simulates gravity, inertia, friction, and collisions.
- **Sensor Simulation**: Provides realistic data for Lidars, Cameras, IMUs, and GPS.
- **World Building**: Allows you to create complex environments (offices, outdoor terrain, factories).
- **ROS 2 Integration**: Gazebo acts like a "Virtual Robot." It subscribes to your velocity commands and publishes sensor data exactly like a real hardware platform.

Why simulate?

Simulation is free and safe. You can crash a virtual robot 1,000 times while debugging your code without breaking expensive hardware or hurting anyone.

How does our ROS 2 code talk to the Gazebo simulation? We use Plugins.

- **Gazebo ROS Plugins**: These are "translators" that sit inside the simulation.
- **Input**: The plugin listens for `geometry_msgs/msg/Twist` (just like the turtle!).
- **Output**: The plugin publishes `nav_msgs/msg/Odometry` and sensor topics.

Code Portability

Because Gazebo uses the same message types as Turtlesim, the controller you wrote can move a 3D differential drive robot with very few modifications!

To work with Gazebo, you will encounter these specific file formats:

- **SDF (Simulation Description Format)**: An XML format used to describe objects and environments (worlds) in Gazebo.
- **URDF (Unified Robot Description Format)**: The ROS-standard way to describe the visual and physical properties of a robot (links and joints).
- **.world files**: These files contain everything: the floor, the lights, the obstacles, and the robots of the generated simulation.

To start Gazebo in a ROS 2 environment, we typically use the `gazebo_ros` package.

```
# 1. Install the Gazebo ROS packages
sudo apt install ros-humble-gazebo-ros-pkgs

# 2. Launch Gazebo (Empty World)
ros2 launch gazebo_ros gazebo.launch.py

# 3. Check the active topics (You will see Gazebo-specific ones)
ros2 topic list
```

Gazebo: The Interface

The Gazebo window is divided into several panels, each serving a specific purpose for simulation management and robot monitoring.

- **The Main Stage (Scene)**: The central 3D view where you interact with the robot and the environment.
- **The Toolbars**: Top-level icons for selecting, translating, and rotating objects.
- **Side Panels**: Used for managing the "World" properties and the "Library" of 3D models.

Mouse Controls

- **Left Click**: Select objects.
- **Scroll Wheel**: Zoom in/out.
- **Middle Click (Hold)**: Pan the camera.
- **Right Click (Hold)**: Orbit/Rotate the camera.

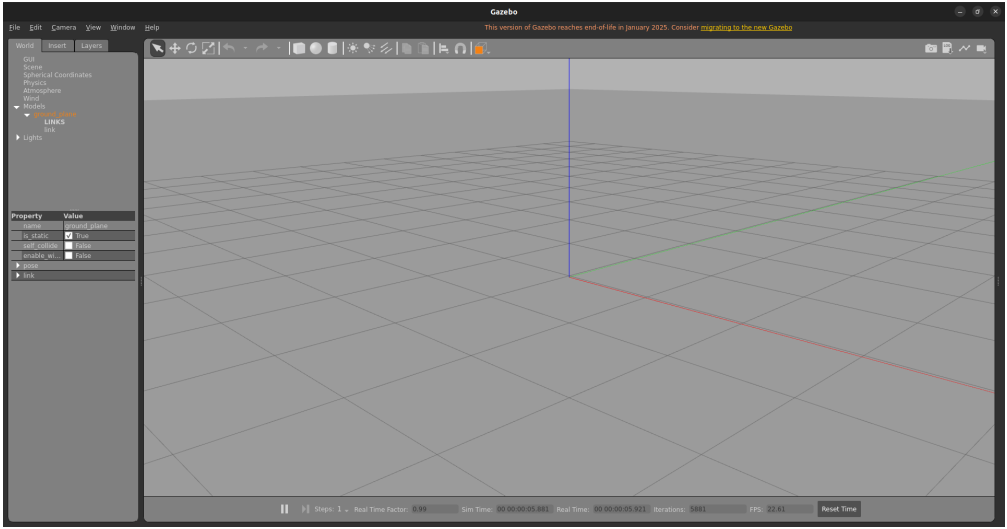


Figure: Overview of the Gazebo Simulator Interface. X = Red, Y = Green, Z = Blue (Memory trick RGB → XYZ).

To control the "Physics" and the "Models," you need to master these three tabs on the left sidebar:

- **World Tab**: Lists every object currently in the simulation (the robot, lights, ground plane, obstacles). You can click them to see their Pose (x, y, z).
- **Insert Tab**: A library of 3D models. You can "drag and drop" walls, furniture, or other robots directly into your world.
- **Layers**: Allows you to hide/show specific groups of objects to simplify the view.

Physics Control (Bottom Bar)

Metric	Description
Real Time Factor (RTF)	How fast the sim runs vs. reality (1.0 = Real time).
Sim Time	The "internal clock" of the simulation.
Iterations	How many physics steps have been calculated.

The "Reset" Button

If your robot flips over or flies away due to a physics glitch, use the **Reset World** button (usually in the bottom bar) to bring everything back to its starting pose.

Populating the World

The **URDF** (Unified Robot Description Format) is an XML file that describes the entire physical makeup of the robot.

- **Links**: The "bones" (Chassis, wheels, sensor housings).
- **Joints**: The "connections" (Where wheels rotate or where a Lidar is mounted).
- **Visual**: How the robot looks (3D meshes).
- **Collision**: The simplified shape used for physics calculations.
- **Inertial**: The mass and weight distribution (how heavy is the robot?).

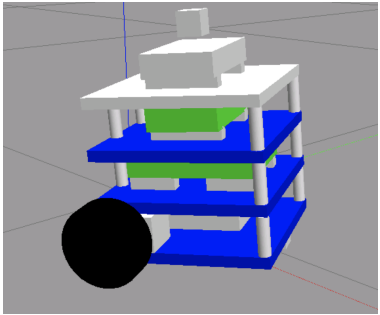


Figure: Collision shapes rendered in Gazebo.

XML Structure

Every URDF follows a tree structure. There is a "Root" link (usually `base_...`) and everything else is attached to it via joints.

A simplified TurtleBot3 wheel .urdf

```
<link name="wheel_left_link">
  <visual>
    <geometry>
      <cylinder length="0.018" radius="0.033"/>
    </geometry>
  </visual>
  <collision>
    <geometry>
      <cylinder length="0.018" radius="0.033"/>
    </geometry>
   </collision>
</link>
<joint name="wheel_left_joint" type="continuous">
  <parent link="base_link"/>
  <child link="wheel_left_link"/>
  <origin xyz="0 0.08 0.023" rpy="-1.57 0 0"/>
  <axis xyz="0 0 1"/>
</joint>
```

Continuous vs. Fixed

A wheel uses a **continuous** joint (it spins). The Lidar uses a **fixed** joint (it never moves relative to the chassis).

Xacro (XML Macros) is a scripting language for URDFs that makes them shorter and more readable.

- **Constants**: Define a wheel radius once: `<xacro:property name="radius" value="0.033"/>`.
- **Macros**: Create a "Wheel" template and use it for both the left and right sides.
- **Math**: Perform calculations directly in the file (e.g., `$pi / 2`).

Converting Xacro to URDF

```
# ROS 2 processes Xacro on the fly during launch:
xacro robot.urdf.xacro > robot.urdf
```

Why use it?

If you decide to change the size of your robot's wheels, you only change **one** value in a Xacro file, instead of searching through 500 lines of a standard URDF.

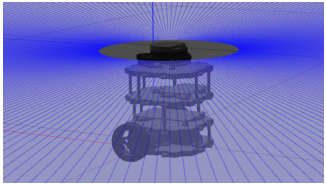
In this lab, we will use the **TurtleBot3 Burger**. Before Gazebo can "see" the robot, we must tell ROS 2 which model we are using.

1. **Set the Model**: Define the environment variable in your terminal.

```
sudo apt update # Update your package list
sudo apt install ros-humble-turtlebot3-gazebo # Install TurtleBot3 Gazebo sim.
source /opt/ros/humble/setup.bash
export TURTLEBOT3_MODEL=burger
```

2. **Launch the Simulation**: Start Gazebo with the robot already inside an empty world.

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```



Adding Objects:

- Use the **Insert Tab** to add cubes, spheres, rooms...
- These act as **Obstacles** for the robot's Lidar sensor.

Environment Variables

If you forget the `export` command, ROS 2 won't know which URDF file to load, and the simulation will fail to spawn the robot!

The "DNA" of our robot is located in: `/opt/ros/humble/share/turtlebot3_description/urdf/` You can manually add that system folder to your current VS Code window so you can browse the URDFs side-by-side with your code:

- Go to File → Add Folder to Workspace...
- Navigate to /opt/ros/humble/share/turtlebot3_description (or just to /opt)
- Click Add.

Now you will see a second "root" folder in your sidebar called turtlebot3_description. You can find the URDFs under urdf/. **Key Sections to Look For:**

- **Joint** `base_joint`: Connects the footprint (ground) to the chassis.
- **Link** `base_scan`: Defines where the Lidar sits.
- **Gazebo Sensors**: Special tags starting with <gazebo> that tell the simulator to generate "fake" Lidar data for that link.

A <link> defines a single "part" of the robot. In the code you see:

- <visual>: What the robot looks like in Gazebo. It uses **.stl meshes** for high detail or simple shapes (box, cylinder).
- <collision>: The "invisible" shield used for physics. Note that this is often a **simpler shape** than the visual mesh to save CPU power!
- <inertial>: Defines the **Mass** and the **Inertia Matrix**. This determines how the robot tips, accelerates, and reacts to gravity.

Visual vs. Collision

Look at base_link: The *visual* is a complex mesh, but the *collision* is a simple **box**. This makes the math much faster for the simulator!

```
<link name="...">
    <visual>
      ...
    </visual>
    <collision>
      ...
    </collision>
    <inertial>
      ...
    </inertial>
</link>
```

A <joint> describes how a **Child Link** moves relative to a **Parent Link**.

- type="fixed": The links never move apart (e.g., the Lidar mounted on the chassis).
- type="continuous": Rotates infinitely around an axis (e.g., wheels).
- <origin>: The offset (x, y, z) and rotation (r, p, y) from the parent's center.
- <axis>: Which direction the part rotates (e.g., xyz="0 0 1" is the Z-axis).

The Root: base_footprint

Notice that base_link is a link with **no** geometry. It represents the point on the floor directly under the robot's center. It is the "anchor" for the entire coordinate system.

```
<joint name="..." type="...">
    <parent link="..."/>
    <child link="..."/>
    <origin xyz="..." rpy="..."/>
    <axis xyz="..."/>
</joint>
```

In the TurtleBot3 URDF, we see a few more specialized tags:

- `<material>`: Defines colors (like `dark` or `blue`). These are shared across links to keep the robot's appearance consistent.
- **Meshes**: Instead of defining every vertex, we import `.stl` files from the `turtlebot3_common` package.
- **The Caster Wheel**: Look at `caster_back_link`. It only has collision and inertia. In simulation, this acts like a "frictionless slider" to keep the 2-wheeled robot balanced.

What's Missing?

This URDF tells ROS how the robot **looks** and **moves**. To make it **live**, we would add `<gazebo>` tags to include plugins for the motors and the Lidar.

The TurtleBot moves in Gazebo because of a "Bridge" that connects ROS topics to Gazebo's physics engine.

- **The Plugin**: A library (`libgazebo_ros_diff_drive.so`) attached to the robot model.
- **Input**: It listens to the `/cmd_vel` topic (Twist messages).
- **Action**: It calculates how fast the **left** and **right** wheels should spin to achieve that velocity.
- **Output**: It applies physical **Torque** to the joints in the simulation.

Odometry

The plugin also does the reverse: it measures how much the wheels rotated and publishes the robot's "calculated" position to the `/odom` topic.

When you send a command like `linear.x = 0.2`, the Gazebo plugin performs the following math based on the URDF parameters:

- **Wheel Radius (r)**: Found in the URDF (0.033m).
- **Wheel Separation (L)**: Distance between the two wheels (0.18m).

The Equations:

$$\omega_{left} = \frac{v - (\omega \cdot L/2)}{r}$$

$$\omega_{right} = \frac{v + (\omega \cdot L/2)}{r}$$

Where v is linear velocity and ω is angular velocity.

Physics vs. Turtlesim

In Turtlesim, the turtle simply "teleports" to the next frame. In Gazebo, the wheels must overcome **friction** and **inertia** to move the mass defined in your URDF.

When you launch the TurtleBot3 in Gazebo, the simulator often reads an **SDF** file located in the `models/` folder in `turtlebot3_gazebo`.

- **URDF**: Focused on the robot's structure for ROS nodes.
- **SDF**: Focused on the simulation world. It includes "World" properties like gravity, light, and complex physics plugins.
- **The Connection**: In modern ROS 2, we often use the `gazebo_ros` factory to convert URDF to SDF on the fly, or we point directly to a pre-defined SDF model.

Why SDF?

SDF can describe multiple robots in one file, along with their environment. It is more flexible than URDF for complex physical interactions.

Inside the `model.sdf` file, you will find the `<plugin>` tag. This is the "Software Driver" for your virtual wheels.

- **Command Listener**: It creates a ROS 2 subscriber for `/cmd_vel`.
- **Physics Link**: It maps that topic to the `wheel_left_joint` and `wheel_right_joint` defined in the SDF.
- **Odometry Publisher**: It calculates where the robot is based on wheel rotation and publishes to `/odom` and `/tf`.

Key Parameters

If you change the `<wheel_separation>` or `<wheel_diameter>` in this plugin, your robot will move incorrectly! The plugin must match the physical dimensions of the 3D model.

Even though the Burger is a complex 3D model in an SDF file, the interface remains identical to the 2D Turtle.

1. **Your Node**: Publishes `Twist` to `/cmd_vel`.
2. **The SDF Plugin**: Receives the `Twist` and converts it to **Joint Torque**.
3. **Gazebo Physics**: Applies the torque, considers friction and gravity, and moves the wheels.
4. **Feedback**: The plugin publishes the new position back to ROS.

Hardware Agnostic

This is the power of ROS 2: Your Python script doesn't care if it's talking to a `.py` turtle, an `.sdf` simulation, or a real `C++` hardware driver!

In a simulation, a sensor is composed of two distinct parts:

1. **The Model (SDF/URDF)**: Defines the physical size and position of the sensor on the robot.
2. **The Plugin (The Bridge)**: A piece of code that:
   - Queries the Gazebo physics engine for "hits" (collisions).
   - Packages that data into a standard ROS 2 message.
   - Publishes it to a ROS 2 topic (e.g., `/scan` or `/imu`).

Sensor Noise

Real sensors aren't perfect. Gazebo allows us to add **Gaussian Noise** to simulated sensors to make the lab more realistic and test how robust our code is!

Simulation Time

It's worth emphasizing that for ROS nodes to use Sim Time, they must be started with the parameter `use_sim_time:=True`. If not, their `get_clock().now()` will use the system clock, leading to errors in transformations (TF).

Sensor Types

Sensor	Gazebo Type	ROS 2 Topic / Message
Lidar	`ray`	`/scan` (LaserScan)
Camera	`camera`	`/image_raw` (Image)
IMU	`imu`	`/imu` (Imu)
Odometry	`diff_drive`	`/odom` (Odometry)

The Processing Flow:

`Gazebo Physics` → `Gazebo Plugin` → `ROS Topic` → `Your Python Code`

Inside the `model.sdf`, the Lidar is defined as a `ray` sensor with a ROS plugin:

SDF Sensor Block

```
<sensor name="lds_lfcd_sensor" type="ray">
  <ray>
    <scan>
      <horizontal>
        <samples>360</samples>
        <resolution>1</resolution>
        <min_angle>0.0</min_angle>
        <max_angle>6.28319</max_angle>
      </horizontal>
    </scan>
  </ray>
  <plugin name="turtlebot3_laserscan" filename="libgazebo_ros_ray_sensor.so">
    <ros>
        <remapping>~/out:=scan</remapping>
    </ros>
  </plugin>
</sensor>
```

A URDF describes the *shape* of a sensor, but a Gazebo Plugin makes it *work*. Inside the URDF, you will find a "Gazebo" block. This tells Gazebo: *"Turn this cylinder into a Lidar and publish the data to the /scan topic."*

Once Gazebo is running with the TurtleBot3 World, open a new terminal and run:

```
# 1. Check if the topic exists
ros2 topic list

# 2. Inspect the data structure
ros2 topic info /scan

# 3. See the live numbers (Distance in meters)
ros2 topic echo /scan --field ranges
```

Challenge

Try placing a "Jersey Barrier" in front of the robot using the **Insert Tab** while `echo` is running. Watch how the numbers in the array change in real-time!

Deep Dive: The 4 Key Robot Sensors

Theory: Lidar (Light Detection and Ranging) uses "Time of Flight." It shoots a laser and measures the time it takes to bounce back. By spinning 360°, it creates a 2D map of obstacles.

- **Topic**: `/scan`
- **Type**: `sensor_msgs/msg/LaserScan`

The LaserScan Structure

```
float32[] ranges          # Array of 360 distances (in meters)
float32 angle_min         # Start angle (usually 0)
float32 angle_max         # End angle (usually 6.28)
float32 range_min         # Minimum reliable distance
float32 range_max         # Maximum distance (e.g., 3.5m)
```

Reading the Array

`ranges[0]` is the distance directly in front of the robot. `ranges[90]` is the distance to the left. If a value is `inf`, the area is clear!

Theory: A digital sensor captures light in a grid of pixels. In robotics, we use cameras for "Visual Servoing" (e.g., following a line) or "Object Detection."

- **Topic**: `/image_raw`
- **Type**: `sensor_msgs/msg/Image`

Understanding Image Data

Field	Meaning	Example
`height/width`	Resolution in pixels	640 x 480
`encoding`	Color format	`rgb8` or `bgr8`
`data`	Raw pixel bytes	A giant list of 0-255 values

Bandwidth Warning

Raw images are heavy. Sending a 640x480 RGB image creates a stream of nearly 1MB **per frame**. This is why computer vision nodes often run locally on the robot's CPU.

Theory: Combines an **Accelerometer** (linear force) and a **Gyroscope** (rotational speed). It is the robot's "inner ear," helping it stay balanced and track its heading.

- **Topic**: `/imu`
- **Type**: `sensor_msgs/msg/Imu`
- **Linear Acceleration**: Measures g forces. If the robot hits a wall, you'll see a spike in the X axis.
- **Angular Velocity**: How fast the robot is spinning (rad/s).
- **Orientation**: Represented as a **Quaternion** (x, y, z, w).

The Gravity Vector

Even when the robot is still, the IMU reports $z \approx 9.8\ m/s^2$. This is how the robot knows which way is "Down."

Theory: Odometry estimates the change in position over time based on **Wheel Encoders**. If we know the wheel radius and the number of rotations, we can calculate the distance traveled.

- **Topic**: `/odom`
- **Type**: `nav_msgs/msg/Odometry`

Odom Message Parts

Section	Content	Purpose
Pose	(x, y, z) + Orientation	Where the robot is.
Twist	Linear & Angular Vel.	How fast it is moving.
Frame ID	`"odom"`	The fixed starting point.

The "Drift" Problem

Odometry is not perfect. If the wheels slip on a smooth floor, the robot "thinks" it moved, but it stayed still. We use Lidar and IMU to correct this error (SLAM/EKF).

In Turtlesim, the heading was a simple `float` (θ). In `nav_msgs/msg/Odometry`, the orientation is a **Quaternion** (x, y, z, w).

- **The Error**: Reading `msg.pose.pose.orientation.z` will **NOT** give you the heading in radians. A Quaternion is a 4D complex number used to avoid "Gimbal Lock."
- **The Solution**: We must convert the Quaternion into **Euler Angles** (Roll, Pitch, and **Yaw**).
- **Yaw**: This is the "Z-rotation" (heading) we actually need for 2D navigation.

Calculation

For a robot moving on a flat floor, the conversion from (x, y, z, w) to θ (Yaw) is:

$$\theta = \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2))$$

Lab Task: Gazebo Velocity Control

The logic you used for the turtle can be applied to a 3D robot. However, we must change our "Data Sources" to match real hardware standards.

Topic Remapping

Feature	Turtlesim	Gazebo / TurtleBot3
Control Topic	`/turtle1/cmd_vel`	`/cmd_vel`
Feedback Topic	`/turtle1/pose`	`/odom`
Feedback Type	`turtlesim/msg/Pose`	`nav_msgs/msg/Odometry`

Coordinate System

In Turtlesim, $(0, 0)$ is the bottom-left corner. In Gazebo, $(0, 0)$ is the **center of the world**. Negative coordinates are possible and common!

Unlike the simple `Pose`, the `Odometry` message is nested. To get the position, we must navigate the object: `msg.pose.pose.position.x`.

Gazebo Controller Snippet

```
from nav_msgs.msg import Odometry #!nav_msgs
from geometry_msgs.msg import Twist
...
def odom_callback(self, msg: Odometry):
    x = msg.pose.pose.position.x
    y = msg.pose.pose.position.y
#The first .pose is part of a PoseWithCovariance message (which includes
#uncertainty math), and the second .pose is the actual Pose object.
    cmd = Twist()
    if abs(x) > 2.0: # Boundary check
        cmd.linear.x = 0.1
        cmd.angular.z = 1.0
    else:
        cmd.linear.x = 0.3
    self.cmd_pub.publish(cmd)
```

Units Matter

Always use SI units. `linear.x` is in **meters per second** (m/s), and `angular.z` is in **radians per second** (rad/s).

Objective: Create a node that keeps the TurtleBot3 inside a defined square area in the Gazebo "Empty World".

1. **Setup**: Launch the simulation:
   - `export TURTLEBOT3_MODEL=burger`
   - `ros2 launch turtlebot3_gazebo empty_world.launch.py`
2. **Develop**: Write a node that subscribes to `/odom` and publishes to `/cmd_vel`.
3. **Constraint**: If the robot exceeds $|x| > 1.5$ or $|y| > 1.5$, it must rotate until it faces back toward the origin.

Pro-Tip: Monitoring

Use `ros2 topic echo /odom` in a separate terminal to see your coordinates in real-time while your code is running.

In Turtlesim, if you send `linear.x = 0.0`, the turtle stops **instantly**. In Gazebo, the robot has **Mass** and **Momentum**.

- **Inertia and Friction**: If the Burger is moving at 0.22 m/s and you command 0.0, it will slide forward a few centimeters before stopping.
- **The Overshoot**: If your logic says "Turn at $x = 1.5$", the robot might actually reach $x = 1.52$ before the motors can physically rotate it.

Control Strategy

When designing your "Safe Square" logic, don't use strict equality (`if x == 1.5`). Instead, use **thresholds** (`if x > 1.5`) to account for physics-induced overshoot. Try to verify when the robot stops after the command.

In Turtlesim, $(0, 0)$ was the bottom-left. In Gazebo, we follow the standard for coordinate frames:

- **Map/World Frame**: The fixed "Global" origin $(0, 0, 0)$ of the 3D environment.
- **Odom Frame**: The point where the robot was when the node started.
- **Base Link**: The "Local" frame attached to the robot.

Directional Convention (Right-Hand Rule)

- **+X**: Forward (Red axis)
- **+Y**: Left (Green axis)
- **+Z**: Up (Blue axis)

Rotation is positive Counter-Clockwise (following your right thumb).

Gazebo calculates physics in "steps." If the physics are complex, the simulation might run slower than real life.

- **Real Time**: The clock on your wall/computer.
- **Sim Time**: The clock inside Gazebo's physics engine.

Crucial Parameter for your Nodes

```
# When running nodes with Gazebo, ALWAYS set this:
# ros2 run my_pkg my_node --ros-args -p use_sim_time:=true
```

What happens if you forget?

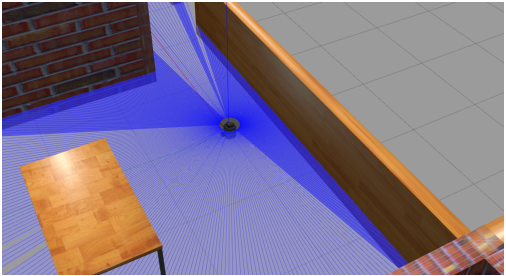
If `use_sim_time` is `False`, your node uses the system clock. If Gazebo is lagging, your transformations (*TF*) and sensor data will be "out of sync," and your controller will fail.

The **World Tab** (left sidebar) lists every "entity" currently alive in the physics engine.

- **Model Tree**: Click on `turtlebot3_burger` to expand its parts (Links and Joints).
- **Property Editor**: When an object is selected, you can view and edit its:
  - **Pose**: Precise x, y, z and Roll, Pitch, Yaw.
  - **Inertia**: The mass and center of gravity (advanced).
  - **Visual/Collision**: Toggle how the robot looks vs. how it "hits" things.

The **Insert Tab** connects to local folders and the Gazebo Model Database online.

- **Local Models**: Basic shapes (Box, Sphere, Cylinder).
- **Fuel / Online Models**: Complex objects like houses, fire hydrants, or kitchen tables.
- **The "Drop" Workflow**:
  1. Click the model name in the list.
  2. Move your mouse to the 3D scene (a ghost image appears).
  3. **Left-click** to place it in the world.



Custom Worlds

You can save your current setup (Robot + Obstacles) as a **.world** file, so you don't have to manually place objects every time you restart the lab.

Gazebo can show you exactly what the robot's sensors are "feeling" in 3D.

- **View Menu** → **Transparent**: Makes the robot's skin see-through to view internal joints.
- **View Menu** → **Contacts**: Shows blue dots where objects are touching (useful for collision debugging).
- **Lidar Visualization**:
  - Go to **World Tab** → `turtlebot3` → `base_scan`.
  - Check the **Visualized** box to see the laser beams in the 3D scene.

Lidar in Gazebo

The Lidar in Gazebo isn't just a graphic; it's a simulated laser. If you place a box in front of the robot, the blue lines will shorten, and the `/scan` topic in ROS 2 will update instantly.

Gazebo does not run on "Wall Clock" time; it runs on **Simulation Time**.

- **Play/Pause Button**: If paused, ROS 2 messages will be received, but the robot will not move.
- **Real Time Factor (RTF)**:
  - **1.0**: The simulation matches real life.
  - $<$ **1.0**: Your computer is struggling with physics; 1 second in sim takes $>$ 1 second in real life.
- **Sim Time**: This is the time sent to ROS 2. If this stops, your Python `timers` also stop!

Inside the .world File

Every simulation starts with a "Default" setup. Even when the world looks empty, the `.world` file is defining the laws of physics.

- `<include>`: Imports external models (The Sun for light, the Ground Plane for a floor).
- `<physics>`: Defines the "Brain" of the simulation (**ODE** - Open Dynamics Engine).
  - `max_step_size`: 0.001s (Calculates physics every 1ms).
  - `real_time_factor`: 1.0 (The sim tries to match real-world speed).
- `<gui>`: Remembers where your camera was pointing last time you saved.

Physics Constraints

The `<constraints>` section (CFM/ERP) defines how "hard" or "squishy" surfaces are. This is why a robot bounces slightly when it hits the floor!

To create a "Stage", we simply take the empty_world template and inject more models.

Added Components:

- **The Room**: turtlebot3_dqn_world
- **The Obstacles**: Fixed blocks for the robot to avoid.
- <static>1</static>: This is crucial! It tells Gazebo these walls **cannot move**, even if the robot hits them at full speed.

Defining the Stage

```
<model name="turtlebot3_dqn_world">
<static>1</static>
<include>
    <uri>model://turtlebot3_dqn_world</uri>
</include>
</model>

<include>
    <uri>model://obstacles</uri>
</include>
```

URI (Uniform Resource Identifier)

The model:// syntax tells Gazebo to look inside your GAZEBO_MODEL_PATH folders to find the actual 3D meshes and collision data.

Think of the `.world` file as a "Theater Script."

World Comparison

Element	Empty World	DQN Stage World
Lighting	Sun (`included`)	Sun (`included`)
Physics	ODE (1ms steps)	ODE (1ms steps)
Environment	Infinite Floor	Enclosed Square Room
Obstacles	None	Static Pillars / Walls
Complexity	Lowest CPU load	Higher (More collisions to calculate)

The Logic for the Lab: Instead of manually adding walls one by one in the Gazebo GUI, we use these pre-made worlds to ensure every student is testing their code on the **exact same map**.

Beyond the Defaults: Custom Worlds

You don't have to build every chair, wall, or robot from scratch. The global ROS community shares thousands of high-quality 3D models online.

- **Gazebo Fuel**: The official "App Store" for Gazebo models.
- **OSRF (Open Source Robotics Foundation)**: They provide standard sets.
- **AWS RoboMaker**: Amazon provides massive open-source worlds.

Popular Web Models

Category	Example	Use Case
Obstacles	`Construction Cone`	Testing Lidar detection.
Furniture	`Office Chair`	Path planning in a room.
Characters	`Walking Person`	Dynamic obstacle avoidance.

If you need a specific floor plan for your laboratory or home, Gazebo has a built-in "Architect" mode.

1. **Edit → Building Editor** (Ctrl + B).
2. **Draw Walls**: Trace a 2D floor plan (like a blueprint), and Gazebo extrudes it into 3D.
3. **Add Textures**: Choose wood, brick, or concrete for the walls and floor.
4. **Save**: Gazebo creates a folder with a `model.sdf` and `model.config` automatically.

Lab Hint

Once you "Finish" the building, it appears in your world. You can then save the entire setup as a new `.world` file to use in your Launch Scripts!

When you create a `model.sdf` file for a simple object (like a custom obstacle), you define its "DNA" using three main components: **Visual**, **Collision**, and **Inertial**.

- **The Shape**: Defined inside the `<geometry>` tag.
- **Primitive Types**:
  - `<box>`: Define `<size>` (x,y,z in meters).
  - `<sphere>`: Define `<radius>`.
  - `<cylinder>`: Define `<radius>` and `<length>`.

A Simple 1m x 1m x 1m Box

```
<visual name="visual">
    <geometry>
        <box>
            <size>1.0 1.0 1.0</size>
        </box>
    </geometry>
</visual>
```

Visual vs. Collision

You must define the geometry for **both**. If you forget the `<collision>` tag, the robot will drive straight through the box like a ghost!

If your custom object is not **Static**, Gazebo needs to know how heavy it is and how its mass is distributed to calculate collisions correctly.

- **Mass**: Weight in kilograms (kg).
- **Inertia (I)**: A matrix defining how hard it is to rotate the object.

Physics for a 5kg Cube

```
<inertial>
<mass>5.0</mass>
<inertia>
<ixx>0.83</ixx> <ixy>0.0</ixy> <ixz>0.0</ixz>
<iyy>0.83</iyy> <iyz>0.0</iyz> <izz>0.83</izz>
</inertia>
</inertial>
```

Simplification Tip

If you just want a wall that never moves, set `<static>true</static>`. This ignores all inertia math and saves a huge amount of CPU power during the lab.

To make your custom shape "discoverable" by ROS 2 and Gazebo, you must follow this specific folder structure inside your package:

- `model.config`: Metadata (Name, version, description).
- `model.sdf`: The XML code defining the shape and physics.
- `meshes/` (Optional): If you decide to add a complex `.stl` or `.dae` file later.

File Purpose

File	Content	Mandatory?
`model.config`	Name of the model + SDF version.	**Yes**
`model.sdf`	Geometry, Collisions, and Inertia.	**Yes**
`materials/`	Colors and Textures (PNG/JPG).	No

Lab Workflow

1. Create the folder. 2. Write the XML. 3. Update `setup.py`. 4. Use the model in your Launch file using the `-file` argument.

When the standard library isn't enough, you have two ways to build your own environment and obstacles.

1. **The Visual Way (Building Editor)**:
   - Open Gazebo and go to `Edit` → `Building Editor` (Ctrl + B).
   - Import a floor plan image or draw walls directly in 2D.
   - Add windows, doors, and textures, then click **Finish** to save the folder.
2. **The Manual Way (XML Scripting)**:
   - Create a new folder in your package (e.g., `models/my_obstacle/`).
   - Write a `model.config` (metadata) and a `model.sdf` (geometry).
   - Define the `<visual>` (how it looks) and `<collision>` (how it hits).

A Minimal model.config Template

```xml
<?xml version='1.0'?>
<sdf version='1.6'>
    <model name='my_obstacle'>
    ...
    </model>
    <static>1</static>
</sdf>
```

Pro Tip

Always set your custom environment models to `<static>1</static>` if they shouldn't move. This prevents the robot from accidentally "pushing" the walls of your house!

Launch Files

Launching a 3D simulation manually is tedious: you would need five terminals for the Server, Client, Robot Spawner, and Obstacles. A **.launch.py** file is a Python script that automates this into a single command.

- **Portability**: Uses `get_package_share_directory` to find files regardless of the username or path.
- **Automation**: Sets environment variables (like disabling audio) to ensure stability.
- **Coordination**: Manages the lifecycle of multiple nodes and processes at once.

Where to put them?

Launch files must be placed in a folder named `launch/` in the root of your package: `~/ros2_ws/src/ItR_lab_pkg/launch/` Don't forget to update your `setup.py` so they are copied to the `install/` folder during build!

By default, `colcon build` ignores your `launch/` folder. You must explicitly tell the build system to "install" these scripts into the **share** directory.

Updating the data_files list in setup.py

```
import os
from glob import glob
# Inside setup(...)

data_files=[
    # ... other resources ...
    (os.path.join('share', package_name, 'launch'),
    glob(os.path.join('launch', '*launch.[pxy][yma]*'))),
],
```

- `os.path.join`: Creates a system-agnostic path to `share/ItR_lab_pkg/launch`.
- `glob`: A searching tool that finds every file ending in `.py`, `.xml`, or `.yaml` inside your `launch/` folder.

ROS 2 does **not** run the code inside your `src/` folder. It runs the "compiled" copies inside the `install/` folder. Follow this 4-step cycle every time you change a launch file:

1. **Build**: Compile your specific package from the workspace root.

```
cd ~/ros2_ws
colcon build --packages-select ItR_lab_pkg
```

2. **Source**: Update your terminal's environment.

```
source install/setup.bash
```

3. **Launch**: Run the simulation.

```
ros2 launch ItR_lab_pkg my_world_2.launch.py
```

Pro Tip: Tab Completion

If you type `ros2 launch ItR_lab_pkg` and press Tab twice, Linux should list all available launch files. If nothing appears, you forgot to **source**!

If the launch command fails, use this checklist to find the error:

- **Folder Name**: Is it `launch` (lowercase)? ROS 2 is case-sensitive.
- **Filename**: Does it end in `.launch.py`? If it's just `.py`, the `glob` pattern in `setup.py` might miss it.
- **Manual Check**: Look into the install folder directly: `ls ~/ros2_ws/install/ItR_lab_pkg/share/ItR_lab_pkg/launch/`

Common Terminal Errors

Error Message	Likely Cause
`Package not found`	You didn't run `source install/setup.bash`.
`No such file or directory`	Typo in the filename or `setup.py` glob.
`AttributeError`	Python syntax error inside the `.launch.py` file.

Every ROS 2 launch file starts by importing tools from the `launch` and `launch_ros` libraries. These provide the "bricks" used to build the simulation.

Standard Imports

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, SetEnvironmentVariable
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
```

- `os`: Standard Python tool to handle file paths.
- `get_package_share_directory`: A ROS 2 utility to find where a package is installed on the system (crucial for portability).
- `LaunchDescription`: The "Container" that holds all your commands.

ROS 2 looks for a specific function name. If you rename this function, the `ros2 launch` command will fail to find your script.

- **The Constructor**: This function acts as a "factory." It gathers all the nodes, environment variables, and included files into one big "List."
- **The Return**: It must return a `LaunchDescription` object containing every action you want to perform.

The Function Structure

```
def generate_launch_description():
    # A. Define your logic and Actions here...
    my_node = Node(...)

    # B. Return the Description object as a list []
    return LaunchDescription([
        my_node,
    ])
```

A general launch file typically uses two ways to start processes. Knowing when to use which is the key to ROS 2 orchestration:

A. The `Node` Action

- **Use**: Starting a specific executable (Python or C++).
- **Analogy**: "I want to run *this* specific program with *these* settings."
- **Required**: `package`, `executable`.

B. `IncludeLaunchDescription`

- **Use**: Running a launch file that exists in another package.
- **Analogy**: "I want to run the *entire* Gazebo engine that OSRF already wrote."
- **Benefit**: Reusability.

Why the distinction?

Use **Node** for your custom controllers or spawners. Use **Include** for complex infrastructure (like Gazebo or Navigation) that involves many nodes working together.

What happens behind the scenes when you run `ros2 launch`?

1. **Parsing**: ROS 2 runs your `generate_launch_description()` and builds a "Tree" of actions.
2. **Parallelism**: It starts all nodes in the list simultaneously. Unlike a script, Node B does not wait for Node A to finish unless you explicitly use an *Event Handler*.
3. **Aggregated Logging**: The terminal shows you the output (`stdout`) from every running node in a single window, prefixed by the node name.
4. **Teardown**: When you hit Ctrl + C, the launch service sends a shutdown signal to every process in the tree at once.

Timing and Race Conditions

Because nodes start simultaneously, a "Spawner" might try to contact "Gazebo" before Gazebo is ready. This is why we often see *"Waiting for service..."* in the terminal logs.

This script automates the startup of the Gazebo engine and the injection of the TurtleBot3 Burger model.

Full Launch Script

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node

def generate_launch_description():
    #Path to the Gazebo Launch
    pkg_gazebo_ros = get_package_share_directory('gazebo_ros')
    pkg_tb3_gazebo = get_package_share_directory('turtlebot3_gazebo')

    #Include the Gazebo Server
    gzserver = IncludeLaunchDescription(PythonLaunchDescriptionSource(os.path.join(pkg_gazebo_ros, 'launch', 'gzserver.launch.py')))

    #Include the Gazebo Client (The GUI)
    gzclient = IncludeLaunchDescription(PythonLaunchDescriptionSource(os.path.join(pkg_gazebo_ros, 'launch', 'gzclient.launch.py')))
    ...
```

Full Launch Script

```
    ...
    #Robot Spawner
    model_path = os.path.join(
    get_package_share_directory('turtlebot3_gazebo'),
    'models', 'turtlebot3_burger', 'model.sdf')
    spawn_robot = Node(
    package='gazebo_ros',
    executable='spawn_entity.py',
    arguments=[
        '-entity', 'burger',
        '-file', model_path,
        '-x', '0.0', '-y', '0.0', '-z', '0.01'],output='screen')

    return LaunchDescription([gzserver, gzclient, spawn_robot])
```

Imports and Paths

```
pkg_gazebo_ros = get_package_share_directory('gazebo_ros')
pkg_tb3_gazebo = get_package_share_directory('turtlebot3_gazebo')
```

- **Dynamic Paths**: We use `get_package_share_directory` because your code will be installed in a different folder than your source code. This function finds the **install** path automatically.
- **Portability**: This ensures the script works on any computer without hardcoding `/home/user/...`.

Including Gazebo

```
gzserver = IncludeLaunchDescription(...)
gzclient = IncludeLaunchDescription(...)
```

- `gzserver`: This starts the "Brain" (physics engine). It has no window and handles gravity and collisions.
- `gzclient`: This starts the "Eyes" (the 3D GUI window).
- **Note**: You can run a simulation with *only* gzserver to save CPU power!

The Spawner Node

```
spawn_robot = Node(
package='gazebo_ros',
executable='spawn_entity.py',
arguments=['-entity', 'burger', '-file', model_path, ...],
output='screen'
)
```

- `spawn_entity.py`: This is a specialized ROS 2 node. It reads your `.sdf` file and sends it to the Gazebo service.
- `-entity 'burger'`: This provides a unique name for the robot in the Gazebo world.
- `-z '0.01'`: We lift the robot 1cm so it "drops" onto the floor. This prevents the physics engine from "fusing" the wheels into the ground plane.

LaunchDescription

```
return LaunchDescription([gzserver, gzclient, spawn_robot])
```

- **Asynchronous Start**: ROS 2 starts all three actions **simultaneously**.
- **Service Handshake**: Because they start at once, you will see the spawner log: *"Waiting for service /spawn_entity"*. It is waiting for the physics server to finish waking up!

The model_path variable is the "Address Book" entry for your robot. We use a dynamic approach to ensure the code works on any computer.

Building the Path

```
model_path = os.path.join(
get_package_share_directory('turtlebot3_gazebo'),
'models', 'turtlebot3_burger', 'model.sdf'
)
```

- get_package_share_directory: This function searches the ROS 2 workspace for the installed turtlebot3_gazebo folder.
- **The Chain**: It navigates into the subfolders: models → turtlebot3_burger → model.sdf.
- **SDF vs. URDF**: While ROS usually uses URDF, Gazebo prefers **SDF** (Simulation Description Format). This file contains the 3D meshes and physics plugins.

Defining the path isn't enough; we need a "Messenger" to carry that file into the simulation. This is the role of the spawn_entity.py node.

The `arguments` list is where you customize the simulation environment without changing the robot's code.

Argument Breakdown

Flag	Value	Result
`-entity`	`'burger'`	Sets the unique name in the World Tab.
`-file`	`model_path`	Tells Gazebo which 3D model to load.
`-x, -y`	`'0.0'`	The horizontal starting position.
`-z`	`'0.01'`	**The Safety Lift**: Spawns the robot slightly above the floor.

How does ROS 2 know where to look?

1. **The Source**: You wrote your package in `src/`.
2. **The Install**: When you run `colcon build`, your models are copied to `install/`.
3. **The Discovery**: `get_package_share_directory` looks inside the `install/` folder.

Student Task

If you get a "File Not Found" error, verify that the package `turtlebot3_gazebo` is installed and that you have **sourced** your workspace: `source /opt/ros/humble/setup.bash`

The `generate_launch_description()` function does not "run" the nodes itself. It is a Generator that returns a list of instructions to the ROS 2 system.

The Return Statement

```
return LaunchDescription([
gzserver,
gzclient,
spawn_robot
])
```

- **The Container**: `LaunchDescription` is a class that holds a list of **Actions** (Nodes, Includes, or Environment Variables).
- **The List** `[]`: Everything inside the square brackets will be executed by the ROS 2 launch engine.
- **Order of Startup**: In ROS 2, items in this list start asynchronously (simultaneously).

Why the Spawner waits

Because they start at the same time, the `spawn_robot` node usually starts before Gazebo is ready. You will see *"Waiting for service..."* in the terminal. This is normal behavior as the spawner waits for the physics server to "wake up."

While Launch 1 was a "Minimum Viable Simulation," Launch 2 introduces Environment Variables for system stability and Multi-Entity Spawning to create a functional testing world.

New Action: SetEnvironmentVariable

```
env_vars = [
SetEnvironmentVariable('LIBGL\_ALWAYS\_SOFTWARE', '1'),
SetEnvironmentVariable('GAZEBO\_AUDIO\_DEVICE', 'none')
]
```

- `LIBGL_ALWAYS_SOFTWARE`: This is the "Compatibility Mode." It forces the CPU to handle 3D rendering. Essential for students using **Virtual Machines** or laptops without dedicated GPUs to prevent Gazebo from crashing.
- `GAZEBO_AUDIO_DEVICE`: Tells Gazebo to ignore the sound card. This removes the "ALSA" driver warnings that often clutter the terminal during a lab session.

Instead of a blank void, we are now "building" the room by injecting existing models from the `turtlebot3_gazebo` package.

The Walls Node

```
walls_path = os.path.join(pkg_tb3_gazebo, 'models',
'turtlebot3_dqn_world/inner_walls', 'model.sdf')

spawn_walls = Node(
package='gazebo_ros',
executable='spawn_entity.py',
arguments=['-entity', 'walls', '-file', walls_path, '-x', '1.5',
'-y', '0.0', '-z', '0.01'],
output='screen'
)
```

- **Sub-folders**: Note how we navigate into `turtlebot3_dqn_world/inner_walls`.
- **Coordinate Offset**: By setting `-x 1.5`, we place the wall right in front of the robot's Lidar sensor, providing immediate data for your algorithms.

We can repeat the `Node` pattern to add as many objects as we need. Each object acts as an independent "Entity" in the physics engine.

The Obstacle Node

```
obs_path = os.path.join(pkg_tb3_gazebo, 'models',
'turtlebot3_dqn_world/obstacle1', 'model.sdf')

spawn_obs = Node(
package='gazebo_ros',
executable='spawn_entity.py',
arguments=['-entity', 'obs', '-file', obs_path, '-x', '1.5',
'-y', '0.0', '-z', '0.01'],
output='screen'
)
```

- **Unique Entities**: Notice the `-entity` name is `'obs'`. If you used `'walls'` again, Gazebo would reject the second model because the ID is already taken.
- **Practicality**: This method is better than a `.world` file because you can "turn off" obstacles just by commenting out this block in Python.

The final part of the script uses Python list arithmetic (+) to combine the environment settings with our simulation nodes.

Combining Lists

```
# env_vars is a list of [SetEnv1, SetEnv2]
# The rest is a list of [Node1, Node2, ...]

return LaunchDescription(env_vars + [
gzserver,
gzclient,
spawn_robot,
spawn_walls,
spawn_obs
])
```

- **Priority**: Setting `env_vars` first ensures that Gazebo "sees" the software rendering flag the moment it starts.
- **Result**: You get a clean, synchronized startup where the world, the obstacles, and the robot appear in Gazebo simultaneously.

The TurtleBot3 ecosystem relies on a specific "Switch" to know if you are using the **Burger**, **Waffle**, or **Waffle Pi**.

- **Manual Method**: Type `export TURTLEBOT3_MODEL=burger` in every terminal.
- **Automated Method**: Use `SetEnvironmentVariable` in your launch file.

What happens if I forget?

Even if the 3D model appears in Gazebo, the ROS 2 nodes (like `/cmd_vel` or `/scan`) might not link correctly to the physics engine, leaving your robot "paralyzed."

Automation is Key

```
# In your launch file:
SetEnvironmentVariable('TURTLEBOT3_MODEL', 'burger')
```

Advanced Spawning vs. Static Worlds

In ROS 2, you have two ways to build an environment. Your scripts use the Dynamic Spawning method. Here is why it is often the superior choice for development:

- **Static World (.world)**: A hard-coded XML file. To change the position of a box, you must open the file, find the coordinates in 500 lines of XML, and save it.
- **Dynamic Spawning (.launch.py)**: You control the environment via Python variables. Changing a position is as simple as changing a number in your script.

Practicality Comparison

Feature	Static .world File	Dynamic .launch.py
Flexibility	Rigid	Extremely High
Parametrization	Hard (Manual XML edits)	Easy (Python math/loops)
Randomization	Impossible	Possible (e.g., random obstacles)
Portability	Paths often break	`get_share_directory` fixes paths

Because your launch file is Python, you can use **Loops** to create complex patterns that would take hours to write in a `.world` file.

Example: Spawning a Row of Pillars

```
Instead of writing 10 spawn nodes, use a loop!

for i in range(5):
spawn_obs = Node(
package='gazebo_ros',
executable='spawn_entity.py',
arguments=['-entity', f'obs_{i}', '-file', obs_path,
'-x', str(1.0 + i), '-y', '0.0'],
)
ld.add_action(spawn_obs)
```

Practical Application

This is how "Stress Tests" are built. You can instantly generate a forest of 50 obstacles to see if your Lidar algorithm can handle high-density data.

In your second script, you included `SetEnvironmentVariable('LIBGL_ALWAYS_SOFTWARE', '1')`. This is a lifesaver for student laptops.

- **The Problem**: Gazebo is GPU-intensive. On many laptops (especially with integrated Intel/AMD graphics) or Virtual Machines, Gazebo crashes because it can't access the 3D hardware correctly.
- **The Solution**: This variable forces the CPU to handle the graphics (**Software Rendering**).
- **Result**: The simulation is slower, but it is **stable** and won't crash your entire OS.

Lab Tip

If a student says "My Gazebo is just a black screen," check if this environment variable is active in their launch file!

Your scripts use `spawn_entity.py`. It is important to understand that this node is **Transient**.

1. It starts up and connects to the ROS 2 Graph.
2. It looks for the `/spawn_entity` service provided by Gazebo.
3. It sends the SDF/URDF "blueprint" to the physics engine.
4. **It shuts down immediately** once Gazebo confirms the object is created.

The "Blocking" Trap

If the house/walls are huge, Gazebo takes time to calculate the initial physics. If you don't see the robot, look at the terminal: if the script is still "Waiting for service," Gazebo is still busy building the house!

Hands-On Challenges

Goal: Master the Coordinate System and Spawner Logic.

- **Task**: Modify `my_world.launch.py` to create a "Slalom" course.
- **Requirements**:
  1. Spawn **three** separate obstacles.
  2. Place them at: A(2.0,0.5), B(3.5,-0.5), and C(5.0,0.0).
  3. Ensure each has a **unique** `-entity` name in the arguments.
- **Challenge**: Use a Python `for` loop in the launch file to spawn a row of 5 pillars automatically.

Validation

Run your launch file. Open the Gazebo **World Tab** and verify that all 5+ entities appear in the list and are physically separated in the 3D view.

Goal: Use Lidar data (`/scan`) to prevent physical damage.

- **Task**: Create a node `safety_stop.py` that drives the Burger forward but stops before a collision.
- **Logic**:
  1. Subscribe to `/scan` (sensor_msgs/msg/LaserScan).
  2. If **any** value in the `ranges` array is <1.0 meters, publish opposite direction velocity to `/cmd_vel`.
  3. Try publishing zero velocity to stop the robot before moving in opposite direction.

Goal: Combine Odometry and Lidar for autonomous behavior.

- **Task**: Expand your safety node into an "Escape" behavior.
- **Behavior Tree**:
  1. **Drive** until an obstacle is detected (eg. <0.5m).
  2. **Stop** and wait for 1 second.
  3. **Rotate** 90 degrees (check your Yaw from `/odom`).
  4. **Resume** driving in the new direction.

Yaw Calculation Reminder

Remember the Quaternion Trap! Use `atan2` logic to convert the `/odom` orientation into a single angle θ for your rotation logic.

Objective: Reach a coordinate (x,y) in a room filled with random obstacles.

- **The Setup**: Use your `my_world.launch.py` to spawn the robot and at least 3 obstacles.
- **Target**: Set a goal at **x=4.0,y=0.0**.
- **The Constraints**:
  1. The robot must move toward the goal using `/odom`.
  2. If the Lidar (`/scan`) detects an object $< \alpha$m in front, the robot must stop, rotate until the path is clear, and then recalculate the path to the goal.
  3. **Success**: Stop within 0.1m of the target without any collisions.

Logic Tip: The Vector Field

Think of the Goal as a Magnet (Attraction) and the Obstacles as Anti-Magnets (Repulsion). Your resulting velocity should be the sum of these two forces!